



Deadline: March 13, 2026 6:00 PM

Spring 2026

Date: March 10, 2026

1 Comparator with Student Objects

```
1 import java.util.*;
2
3 public class StudentSorter {
4
5     static class Student {
6         String firstName;
7         String lastName;
8         int id;
9
10        public Student(String firstName, String lastName, int id) {
11            this.firstName = firstName;
12            this.lastName = lastName;
13            this.id = id;
14        }
15
16        public String toString() {
17            return id + " | " + firstName + " " + lastName;
18        }
19    }
20
21    public static void main(String[] args) {
22
23        List<Student> students = new ArrayList<>();
24        students.add(new Student("Alice", "Smith", 1003));
25        students.add(new Student("Bob", "Johnson", 1001));
26        students.add(new Student("Charlie", "Smith", 1005));
27        students.add(new Student("Diana", "Adams", 1002));
28        students.add(new Student("Ed", "Brown", 1004));
29        students.add(new Student("Fiona", "Adams", 1006));
30
31        // Comparator 1: by lastName, then firstName
32        Comparator<Student> byLastThenFirst = (a, b) -> {
33            int cmp = a.lastName.compareTo(b.lastName);
34            if (cmp != 0) return cmp;
35            return a.firstName.compareTo(b.firstName);
36        };
37
38        // Comparator 2: by length of firstName
39        Comparator<Student> byFirstNameLength = (a, b) ->
40            Integer.compare(a.firstName.length(), b.firstName.length());
41
42        // Comparator 3: by student ID
43        Comparator<Student> byID = (a, b) ->
44            Integer.compare(a.id, b.id);
45
46        Collections.sort(students, byLastThenFirst);
47        System.out.println("--- Sorted by Last Name then First Name ---");
48        for (Student s : students) System.out.println(s);
49
50        Collections.sort(students, byFirstNameLength);
51        System.out.println("\n--- Sorted by First Name Length ---");
52        for (Student s : students) System.out.println(s);
53
54        Collections.sort(students, byID);
55        System.out.println("\n--- Sorted by Student ID ---");
56        for (Student s : students) System.out.println(s);
57    }
58 }
```

2 Task Scheduling

```
1 import java.util.*;
2
3 public class PQCustomOrders {
4
5     static class Task {
6         String name;
7         int priority;
8         int deadline;
9
10        public Task(String name, int priority, int deadline) {
11            this.name = name;
12            this.priority = priority;
13            this.deadline = deadline;
14        }
15
16        public String toString() {
17            return name + " (priority=" + priority + ", deadline=" + deadline + ")";
18        }
19    }
20
21    public static void main(String[] args) {
22
23        // Queue A: sort by priority first, then deadline
24        Comparator<Task> cmpA = (a, b) -> {
25            int cmp = Integer.compare(a.priority, b.priority);
26            if (cmp != 0) return cmp;
27            return Integer.compare(a.deadline, b.deadline);
28        };
29
30        // Queue B: sort by deadline first, then priority
31        Comparator<Task> cmpB = (a, b) -> {
32            int cmp = Integer.compare(a.deadline, b.deadline);
33            if (cmp != 0) return cmp;
34            return Integer.compare(a.priority, b.priority);
35        };
36
37        PriorityQueue<Task> queueA = new PriorityQueue<>(cmpA);
38        PriorityQueue<Task> queueB = new PriorityQueue<>(cmpB);
39
40        Task[] tasks = {
41            new Task("Email Report", 2, 5),
42            new Task("Fix Bug", 1, 3),
43            new Task("Team Meeting", 2, 2),
44            new Task("Deploy Update", 1, 7),
45            new Task("Write Tests", 3, 1),
46            new Task("Code Review", 3, 4)
47        };
48
49        for (Task t : tasks) {
50            queueA.add(t);
51            queueB.add(t);
52        }
53
54        System.out.println("--- Queue A: Priority first, then Deadline ---");
55        while (!queueA.isEmpty()) System.out.println(queueA.poll());
56
57        System.out.println("\n--- Queue B: Deadline first, then Priority ---");
58        while (!queueB.isEmpty()) System.out.println(queueB.poll());
59    }
60 }
```

3 Hospital Emergency Room

```
1 import java.util.*;
2
3 public class EmergencyRoom {
4
5     static class Patient {
6         String name;
7         int severity;
8         int arrivalTime;
9
10        public Patient(String name, int severity, int arrivalTime) {
11            this.name = name;
12            this.severity = severity;
13            this.arrivalTime = arrivalTime;
14        }
15
16        public String toString() {
17            return name + " (severity=" + severity + ", arrival=" + arrivalTime + ")";
18        }
19    }
20
21    public static void main(String[] args) {
22
23        // Higher severity first; tie earlier arrival first
24        Comparator<Patient> cmp = (a, b) -> {
25            int cmp1 = Integer.compare(b.severity, a.severity); // descending severity
26            if (cmp1 != 0) return cmp1;
27            return Integer.compare(a.arrivalTime, b.arrivalTime); // ascending arrival
28        };
29
30        PriorityQueue<Patient> er = new PriorityQueue<>(cmp);
31
32        er.add(new Patient("Alice", 7, 1));
33        er.add(new Patient("Bob", 9, 2));
34        er.add(new Patient("Charlie", 7, 3));
35        er.add(new Patient("Dina", 10, 4));
36
37        System.out.println("--- Treatment Order ---");
38        while (!er.isEmpty()) {
39            System.out.println("Treating: " + er.poll());
40        }
41
42        // Expected output:
43        // Treating: Dina      (severity=10, arrival=4)
44        // Treating: Bob      (severity=9, arrival=2)
45        // Treating: Alice    (severity=7, arrival=1)
46        // Treating: Charlie  (severity=7, arrival=3)
47    }
48 }
```

4 Priority Queue using Binary Search Tree

```
1 public class BSTpriorityQueue {
2
3     static class Node {
4         String value;
5         int priority;
6         Node left, right;
7
8         public Node(String value, int priority) {
9             this.value = value;
10            this.priority = priority;
11            this.left = null;
12            this.right = null;
13        }
14
15        public String toString() {
16            return value + " (priority=" + priority + ")";
17        }
18    }
19
20    static class PriorityQueueBST {
21
22        Node root;
23
24        public PriorityQueueBST() {
25            root = null;
26        }
27
28        // EnQueue: insert into BST ordered by priority
29        // Lower priority number = more important = goes LEFT
30        public void enqueue(String value, int priority) {
31            root = insert(root, value, priority);
32        }
33
34        private Node insert(Node node, String value, int priority) {
35            if (node == null) return new Node(value, priority);
36            if (priority < node.priority)
37                node.left = insert(node.left, value, priority);
38            else
39                node.right = insert(node.right, value, priority);
40            return node;
41        }
42
43        // DeQueue: remove and return the leftmost node (lowest priority number = highest priority)
44        public Node dequeue() {
45            if (root == null) return null;
46            Node min = findMin(root);
47            root = removeMin(root);
48            return min;
49        }
50
51        private Node findMin(Node node) {
52            while (node.left != null) node = node.left;
53            return node;
54        }
55
56        private Node removeMin(Node node) {
57            if (node == null) return null;
58            if (node.left == null) return node.right; // this is the min, replace with right child
59            node.left = removeMin(node.left);
60            return node;
61        }
62
63        // Peek: return highest priority node without removing
64        public Node peek() {
65            if (root == null) return null;
66            return findMin(root);
67        }
68
69        public boolean isEmpty() {
70            return root == null;
71        }
72    }
73
74    public static void main(String[] args) {
75
```

```

76     PriorityQueueBST pq = new PriorityQueueBST();
77
78     pq.enqueue("Fix critical bug", 1);
79     pq.enqueue("Write report", 4);
80     pq.enqueue("Team meeting", 2);
81     pq.enqueue("Reply to emails", 5);
82     pq.enqueue("Deploy to staging", 2);
83     pq.enqueue("Code review", 3);
84
85     System.out.println("Peek (highest priority): " + pq.peek());
86
87     System.out.println("\n--- DeQueue order (highest priority first) ---");
88     while (!pq.isEmpty()) {
89         System.out.println("Removed: " + pq.dequeue());
90     }
91
92     // Expected output:
93     // Peek (highest priority): Fix critical bug (priority=1)
94     // Removed: Fix critical bug (priority=1)
95     // Removed: Team meeting (priority=2)
96     // Removed: Deploy to staging (priority=2)
97     // Removed: Code review (priority=3)
98     // Removed: Write report (priority=4)
99     // Removed: Reply to emails (priority=5)
100 }
101 }

```